

Guía del mantenedor

Publicar versiones, firmar el plugin y probar

FAVERVIEW versión 3.2.0

Para quien mantiene FAVERVIEW: flujo de trabajo con Pull Requests, versiones, actualizaciones, firma del plugin, pruebas y manuales.

Contenido

1. Para quién es esta guía
2. Cómo está organizado el repositorio
3. Reglas del proyecto
4. Flujo de trabajo: rama, pruebas y Pull Request
5. Publicar una versión nueva (y cómo llega a los usuarios)
6. Firmar y publicar el plugin de Illustrator
7. Pruebas y bancos de pruebas
8. Regenerar los manuales y las capturas
9. Privacidad: lo que nunca se sube
10. Migración del plugin a UXP

1. Para quién es esta guía

Para quien mantiene FAVERVIEW: corrige errores, agrega funciones, publica versiones y firma el plugin. Los usuarios no la necesitan. Versión documentada: 3.2.0. Repositorio: <https://github.com/sebastiansantos0311-dev/faverview>

2. Cómo está organizado el repositorio

Carpeta / archivo	Contenido
app/	Servidor FastAPI: <code>core/</code> (común), <code>modules/</code> (comparar, separar, vectorizar, preflight, códigos, herramientas, automatizar, plugin), <code>learning/</code> (OCR que aprende).
web/	Interfaz (HTML/JS sin compilación): <code>core/</code> y un archivo por módulo.
plugin/	Panel CEP para Illustrator (<code>cep/</code>), pruebas, herramientas de empaquetado, pruebas manuales y migración a UXP.
bench/	Bancos de pruebas: comparar, separación, vectorizador y trapping.
tests/	Pruebas automáticas (pytest) y casos sintéticos.
tools/manual/	Generador de los manuales PDF y capturas.
docs/	Manuales PDF generados.
PLAN*.md, DECISIONES.md, CHANGELOG.md	Planes por etapas, decisiones técnicas y registro de cambios.

3. Reglas del proyecto

- Todo local, gratis y sin IA en la nube. Licencia AGPL-3.0; solo dependencias compatibles.
- Interfaz, mensajes y documentación en español.
- Sin `.exe`, `.bat` ni `.ps1` propios: distribución con GitHub + winget + uv.
- Nunca afirmar «certificado» ni «mejor que X» sin medirlo. Las estimaciones se rotulan como tales.
- Sin bibliotecas Pantone ni otras con licencia.

4. Flujo de trabajo: rama, pruebas y Pull Request

1. Trabaja en una rama: `git switch -c mejora/nombre`. Los usuarios se actualizan desde `main`: **nunca** subas a `main` algo sin probar.
2. Antes de subir, ejecuta todo:

```
uv run pytest -q
uv run python -m bench.run --sinteticos --ci --workers 4
uv run python -m bench.separation.run --ci
uv run python -m bench.vector.run --casos 12 --ci
uv run python -m bench.trapping.run --ci
uv run python plugin/tools/build_zxp.py --sin-firma
```

3. Sube la rama y abre el Pull Request: `git push -u origin mejora/nombre` y `gh pr create` (o desde la web de GitHub).
4. GitHub Actions ejecuta lo mismo en Windows. Solo fusiona el PR con la **marca verde**.
5. Al fusionar a `main`, todos los equipos verán el aviso de actualización (como máximo en 24 horas, o al pulsar «Buscar actualizaciones»).

5. Publicar una versión nueva

1. Sube el número en `pyproject.toml` (`version = "x.y.z"`): x = cambio grande o incompatible, y = función nueva, z = arreglo. El plugin toma la misma versión al empaquetarse.
2. Escribe las novedades en `CHANGELOG.md` en una sección nueva **arriba del todo** (`## x.y.z - fecha`) con viñetas cortas: **las primeras viñetas son las que ve el usuario** en el desplegable «Novedades» del aviso de actualización.
3. Regenera los manuales (sección 8) y ejecuta las pruebas (sección 7).
4. Pull Request → marca verde → fusionar.
5. Etiqueta la versión: `git switch main; git pull; git tag vx.y.z; git push --tags`.
6. Crea el **Release** en GitHub desde la etiqueta (`gh release create vx.y.z --notes-file` o desde la web) y adjunta el plugin firmado (sección 6).

Cómo detecta la app la versión nueva

`app/updates.py` hace `git fetch origin main` una vez al día (o al pulsar «Buscar actualizaciones»), cuenta los commits nuevos, lee la versión de `pyproject.toml` y la primera sección de `CHANGELOG.md` en `origin/main`. «Actualizar» ejecuta `git pull --ff-only` solo si la carpeta está en `main` y sin cambios locales. El plugin busca en los Releases un archivo llamado exactamente `FAVERVIEW-Illustrator-<versión>.zxp`.

6. Firmar y publicar el plugin de Illustrator

1. Descarga **ZXPSignCmd** (herramienta oficial de Adobe, repositorio `Adobe-CEP/CEP-Resources`). No se incluye en el repositorio.
2. Crea **una sola vez** el certificado propio y guárdalo **fuera del repositorio** (`*.p12` está en `.gitignore`):

```
$env:ZXP_CERT_PASS = "una-clave-larga"
$zxp = "C:\herramientas\ZXPSignCmd.exe"
$cert = "C:\seguro\cert.p12"
```

```
uv run python plugin/tools/build_zxp.py --crear-certificado $cert --zxpsigncmd $zxp \
  --pais CO --provincia Antioquia --organizacion "Mi taller" --nombre "FAVERVIEW"
```

3. Empaqueta, firma con sello de tiempo y verifica:

```
$env:ZXP_CERT_PASS = "una-clave-larga"
$zxp = "C:\herramientas\ZXPSignCmd.exe"
$cert = "C:\seguro\cert.p12"
uv run python plugin/tools/build_zxp.py --cert $cert --zxpsigncmd $zxp
```

Sale `build/zxp/FAVERVIEW-Illustrator-<versión>.zxp`.

- Adjúntalo al Release de la versión. Con el mismo nombre de archivo, el panel de los usuarios lo detecta como actualización.
- Guarda una copia de seguridad del certificado y su clave: si los pierdes, las versiones siguientes se firmarán con otro certificado.

El `.zxp` sin firma (`--sin-firma`, el que genera la CI) **no** sirve para instalar con el instalador de Creative Cloud. Para desarrollo usa `uv run python plugin/tools/dev_install.py` (activa PlayerDebugMode, pide confirmación).

7. Pruebas y bancos de pruebas

Qué	Comando	Mide
Pruebas automáticas	<code>uv run pytest -q</code>	Todo el servidor, la capa de Illustrator simulada y el panel.
Banco Comparar	<code>uv run python -m bench.run --sinteticos --ci</code>	Precisión, recall, falsos positivos, CER, tiempo.
Banco Separación	<code>uv run python -m bench.separation.run --ci</code>	Paleta, regiones, ΔE de simulación.
Banco Vectorizador	<code>uv run python -m bench.vector.run --ci</code>	Fidelidad, nodos, huecos contra VTracer/Potrace (e Illustrator/Corel si hay archivos).
Banco Trapping	<code>uv run python -m bench.trapping.run --ci</code>	Filetes tras la prueba de movimiento (deben ser 0).
Plugin en Illustrator	<code>plugin/PRUEBAS_MANUALES.md</code>	13 pruebas manuales en Illustrator real.

Los umbrales de la CI están en `bench/umbral_ci.json` y `bench/*/umbral_ci.json`. Los reportes quedan en `datos_locales/bench_resultados/`.

8. Regenerar los manuales y las capturas

```
uv run python tools/manual/build.py
```

Genera todos los PDF de `docs/` con la versión de `pyproject.toml`. El contenido está en `tools/manual/build.py` (Manual de uso, parte I), `capitulos_suite.py` (parte II) y `documentos.py` (instalación, plugin, guía rápida y esta guía). Para rehacer las capturas (con FAVERVIEW abierto y

Edge instalado):

```
uv run python tools/manual/capturas.py http://127.0.0.1:8000
uv run python tools/manual/capturas_plugin.py
```

9. Privacidad: lo que nunca se sube

- `data/`, `datos_locales/` (casos reales, aprendizaje, tintas, perfiles, plantillas, reportes), `.venv/`, `build/`, certificados `*.p12`.
- Archivos reales de clientes: nunca en `samples/` ni en `tests/` (esas carpetas son públicas). Los casos públicos se generan sintéticamente.
- Antes de cada push: `git status` y revisar que no aparezca nada de lo anterior.
- Los commits usan el correo anónimo de GitHub configurado en este repositorio.

10. Migración del plugin a UXP

Adobe publicará UXP para Illustrator (beta en primavera de 2027) y desactivará CEP en diciembre de 2028. El panel ya está preparado (sin Node, todo el acceso a Illustrator detrás de un adaptador). El mapa de migración está en `plugin/MIGRACION_UXP.md`: cuando salga la beta, crear `PLAN_PLUGIN_UXP.md` y migrar antes de diciembre de 2028.
